AI Real Estate Investment Analyzer — Data & System Design

Overview

This document turns the 11 modules from the FYP proposal into two concrete deliverables: exactly what data each module needs, and how to architect and build the system end to end.

Several modules (ARV prediction, comps, rental estimation, neighborhood analysis) can start from data that already exists publicly. Two modules — property damage detection and rehab cost estimation — have no matching off-the-shelf dataset, so this doc also lays out a plan for building that data.

Data Requirements by Module

Each module needs a different shape of data. Modules 7, 10 and 11 need no independent dataset — they compute from the other modules' outputs.

Module	Core Data Needed	Example Fields
1. Property Damage Detection	Labeled property photos, interior + exterior	image, room type, damage type (roof/wall/mold/flooring/kitchen/bathroom), severity, bounding box or mask
2. Rehab Cost Estimation	Historical renovation cost records	property size, damage type, region, material cost, labor rate, actual amount spent
3. Comparable Property Finder	Recently sold/listed nearby properties	address, lat/long, beds, baths, sqft, sale price, sale date, condition
4. ARV Prediction	Historical sale prices + post-renovation features	sqft, beds, baths, lot size, year built, renovation status, sale price, sale date, location

Module	Core Data Needed	Example Fields
5. Rental Income Estimation	Active rental listings	rent price, beds, baths, sqft, location, property type, listing date
6. Deal Profitability Calculator	User-entered deal inputs	purchase price, rehab estimate, holding costs, closing costs, ARV, rent estimate
7. Maximum Offer Calculator	Derived only — no new data	ARV, rehab cost, target margin %
8. Investment Strategy Recommendation	Historical outcomes by strategy	strategy label, ROI, holding period, local cap rate
9. Neighborhood / Area Analysis	Location-level statistics	avg price/sqft, rent trend, school rating, crime index, walkability, price history
10. Property Summary Report	Derived only — aggregates modules 1–9	—
11. Top Deal Ranking	Derived only — aggregates modules 6, 8, 9	profit, ROI, risk score

The practical split: modules 3, 4, 5 and 9 can be trained or looked up from public listing/market data now; modules 1 and 2 need a dataset built specifically for this project, since "renovation condition from a photo" and "actual rehab spend" aren't things public datasets track.

Dataset & Data Source Options

Target market: US only, confirmed. Every source below is US-focused; the Pakistan option has been dropped.

Need	Source	What it gives	Access
Price/ARV training data	Kaggle: House Prices — Advanced Regression, Ames Housing set	~1,500–2,900 labeled sales with features + price	Free, CSV
Area price & rent trends	Zillow Research Data (ZHVI/ZORI)	Median home value and rent index by region, monthly, historical	Free, CSV
Comps + rental comps + valuation	RentCast API	Property records, rent estimates, sale comps, market stats by address	Free tier (50 calls/mo), paid beyond that
Comps + property data	ATTOM Data, HouseCanary (see real estate API roundup)	Nationwide property, valuation, and comps data	Paid, free trial
Neighborhood: schools	GreatSchools NearbySchools API	School ratings near an address	Commercial license required
Neighborhood: walkability	Walk Score API	Walk/transit/bike score by address	Free tier for low volume
Property damage / condition (closest available)	xBD dataset (disaster damage, satellite), Roboflow Universe interior datasets, Zillow Indoor Dataset	Room-type classification and disaster damage — useful for transfer learning, not a direct match	Free, research use

Build-your-own plan for modules 1 & 2 (no public dataset fits directly): scrape listing photos from Zillow, Realtor.com or Redfin (respecting each site's terms of use) or use a small self-collected photo set; manually label each photo with room type and a condition tier (0 = move-in ready, 1 = cosmetic wear, 2 = moderate repair, 3 = major repair) using a free tool like Roboflow or Label Studio; start the CV model from a pretrained CNN (ResNet/EfficientNet) and fine-tune on this labeled set — a few hundred labeled images

per damage category is enough for a working FYP-grade classifier. For rehab cost, pair labeled damage severity with regional per-sqft renovation cost benchmarks (published by local contractors or general construction-cost guides) instead of needing real invoices.

System Architecture

The system separates into four layers: a Next.js frontend, a FastAPI backend that orchestrates everything, a set of ML/CV services that stay independent so they can be retrained or scaled on their own, and a deal engine that holds the plain business-logic formulas (profit, max offer, ranking) with no ML involved.

```
flowchart TD
  U[Investor - Browser] --> FE[Next.js Frontend]
  FE --> API[FastAPI Backend]
  API --> DB[(PostgreSQL)]
  API --> ST[(Image Storage: S3)]
  API --> CV[CV Service:<br/>Damage Detection]
  API --> ML[ML Service:<br/>ARV / Rehab / Rent]
  API --> COMP[Comps Engine]
  API --> NBHD[Neighborhood Service]
  COMP --> EXT[External APIs:<br/>RentCast / Zillow / GreatSchools]
  NBHD --> EXT
  CV --> DEAL[Deal Engine:<br/>Profit / Offer / Strategy / Rank]
  ML --> DEAL
  DEAL --> RPT[Report Generator: PDF]
  RPT --> DB
```

Keeping CV and ML as separate services (rather than functions inside the main API) matters for an FYP timeline: you can build and demo the Deal Engine early using manually entered numbers, then plug in the real models as they're trained, without reworking the rest of the app.

Tech Stack & Data Flow

This matches the stack already named in the proposal, mapped to each layer.

Layer	Technology	Role
Frontend	Next.js, React, Chart.js	Upload UI, dashboards, deal comparison views

Layer	Technology	Role
Backend API	Python FastAPI	Auth, orchestration, REST endpoints
Database	PostgreSQL	Properties, users, analysis results, deal history
Object storage	AWS S3 (or Cloudinary)	Property photos
CV service	PyTorch, fine-tuned CNN (ResNet/EfficientNet or YOLO)	Damage detection, room condition classification
ML service	Scikit-learn, XGBoost, TensorFlow	ARV prediction, rehab cost estimation, rent estimation (regression)
External data	RentCast, Zillow Research, GreatSchools, Walk Score	Comps, rental comps, school and walkability data
Reporting	WeasyPrint / ReportLab	One-page PDF property reports
Deployment	Docker, AWS (or Render)	Containerized services

Request flow, upload to report:

1. Investor enters a property address and uploads photos.
2. Backend stores photos in S3 and metadata in Postgres.
3. CV service analyzes photos and returns a damage list with severity per room.
4. Backend fetches comps, rental listings and neighborhood stats from external APIs (cached where possible).
5. ML service predicts ARV, rehab cost (from the damage list) and rental income.
6. Deal Engine computes profit, ROI, max offer, and recommends a strategy.
7. Report Generator compiles the one-page PDF; results are stored for ranking.
8. Investor views a dashboard comparing all analyzed properties, ranked by score.

Development Roadmap

Sized for a typical 24-week, two-semester FYP. Build the Deal Engine on manually entered numbers first, so the app is demoable early, then swap in real models as they're trained.

Phase	Weeks	Focus
0. Setup	1–2	Repo, Next.js/FastAPI scaffolding, Postgres schema, auth
1. MVP core	3–6	Deal Profitability Calculator + Max Offer Calculator on manual inputs, no ML yet — validates the business logic and UI end to end
2. Market data	7–12	Comps Engine + Neighborhood Analysis via RentCast + GreatSchools/Walk Score; ARV prediction model trained on Kaggle/Zillow Research housing data
3. Vision & cost models	13–18	Label the custom photo dataset; train the Property Damage Detection CV model; build Rehab Cost Estimation from its output
4. Decision layer	19–22	Investment Strategy Recommendation, Top Deal Ranking, Property Summary Report (PDF)
5. Evaluation	23–24	Test against real deals, polish, prepare FYP defense

The riskiest phase is 3 — it depends on a dataset you build yourself. Start collecting and labeling photos as early as Phase 1, in parallel, so training data is ready by the time Phase 3 starts.